



Sunbeam Institute of Information Technology
Pune and Karad

Algorithms and Data structures

Trainer - Devendra Dhande

Email – devendra.dhande@sunbeaminfo.com

- organising data inside memory for efficient processing along with operations like add, delete, search, etc which can be performed on data.
- eg stack - push/pop/peek

Physical data structures



Logical data structures

- relation betⁿ data
- stack, queue, tree, graph, heap

- data structures are used to achieve
 - Abstraction
 - data & organization of data is hidden from outside.
 - Abstract Data Type (ADT)
 - Reusability
 - as per need
 - to implement other structures
 - to write few algorithms
 - Efficiency
 - can be measured in two parameters:
 1. time - required to execute
 2. space - required inside RAM

(Basic)

Linear data structures

- data is organised sequentially/ linearly



- data can be accessed sequentially

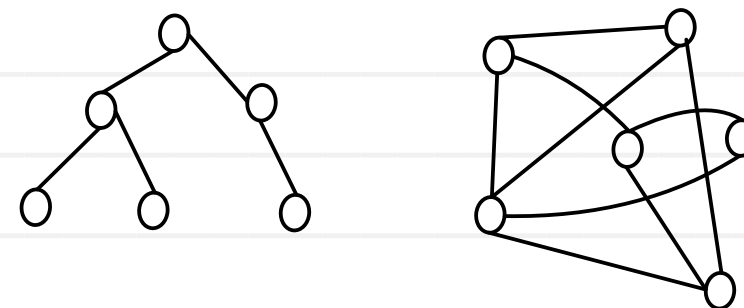
- Array stack
 struct/class queue
 linked list

Hash tables

(Advanced)

Non linear data structures

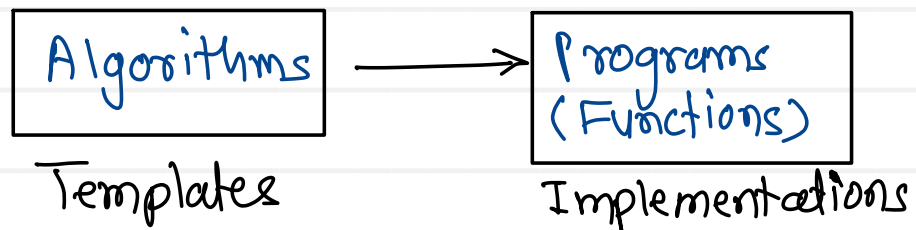
- data is organised in multiple levels (hierarchy)



- data can not be accessed sequentially

- Tree
 Graph
 Heap

- Algorithm is set of instructions given to the programmers
- Algorithm is step by step solution of given problem statement
- As written in human understandable languages, algorithms are programming languages independent
- Algorithms are used as templates and can be implemented in programming languages



Searching – linear/ binary search
Sorting – selection/ bubble/ insertion
quick/ merge/ heap

Problem - Find sum of array elements

Algorithm :

1. create a space to store sum & initialize it to 0.
2. Traverse array from one end to another end ($i = 0 \rightarrow n-1$)
3. Add each data into previously calculated sum.
4. print/return sum

Algorithm analysis

- it is done for efficiency measurement and also known as time/space complexity
- It is done to finding time and space requirements of the algorithm
 1. Time - time required to execute the algorithm (ns, μ s, ms, s, ...)
 2. Space - space required to execute the algorithm inside memory (bytes, KB, MB, ...)
- finding exact time and space of the algorithm is not possible because it depends on few external factors like
 - time is dependent on type of machine (CPU), number of processes running at that time
 - space is dependent on type of machine (architecture), data types

- Approximate time and space analysis of the algorithm is always done
- Mathematical approach is used to find time and space requirements of the algorithm and it is known as “Asymptotic analysis”
- Asymptotic analysis also tells about behaviour of the algorithm for different input or for change in sequence of input
- This behaviour of the algorithm is observed in three different cases
 1. Best case
 2. Average case
 3. Worst case

Time complexity

- time is directly proportional to number of iterations of the loops used in an algorithm
- To find time complexity/requirement of the algorithm count number of iterations of the loops

1. Print 1D array on console

```
void print1DArray (int arr[], int n) {  
    for (i=0; i<n; i++)  
        sysout(arr[i]);  
}
```

No. of iteration = n

Time $\propto$ iterations

Time $\propto n$

$$T(n) = O(n)$$

2. Print 2D array on console

```
void print2DArray (int arr[][], int m, int n)  
{  
    for (i=0; i<m; i++) {  
        for (j=0; j<n; j++) {  
            sysout(arr[i][j]);  
        }  
    }  
}
```

iterations of outer loop = m

iterations of inner loop = n

Total iteration = $m * n$

Time $\propto m * n$

$$T(m, n) = O(m * n)$$

$m \approx n$

Time $\propto n^2$

$$T(n) = O(n^2)$$

3. Add two numbers

```
int add (int n1 , int n2)
    return n1 + n2;
}
```

- constant time requirement

$$T(n) = O(1)$$

4. Print table of given number

```
void printTable (int n) {
    for( i=1 ; i <= 10 ; i++)
        sysout (i * n);
}
```

- constant number of iterations

$$T(n) = O(1)$$

5. Print binary of decimal number

2	9	
	4	1
	2	0
	1	0
		1

$$(9)_{10} = (1001)_2$$

```
while(n > 0) {
    rem = n % 2;
    n = n / 2;
    sysout(rem);
}
```

n	n > 0	rem
9	T	1
4	T	0
2	T	0
1	T	1
0	F	

$$n = n, n/2, n/4, n/8, \dots$$

$$= n/2^0, n/2^1, n/2^2, n/2^3, \dots, n/2^{\text{itr}}$$

for $n=1$, last time loop will be executed

$$n/2^{\text{itr}} = 1$$

$$2^{\text{itr}} = n$$

$$\log 2^{\text{itr}} = \log n$$

$$\text{itr} \log 2 = \log n$$

$$\text{itr} = \frac{\log n}{\log 2}$$

$$\text{Time} \propto \text{itr}$$

$$\text{Time} \propto \frac{\log n}{\log 2}$$

$$T(n) = O(\log n)$$

Time complexity

Time complexities : $O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$, $O(n^2)$, $O(n^3)$,, $O(2^n)$,

Modification : + or - : time complexity is in terms of n

Modification : * or / : time complexity is in terms of $\log n$

$\text{for}(i=0; i < n; i++) \rightarrow O(n)$

$\text{for}(i=n; i >= 0; i--) \rightarrow O(n)$

$\text{for}(i=0; i < n; i+=2) \rightarrow O(n)$

$\text{for}(i=1; i < 100; i++) \rightarrow O(1)$

$\text{for}(i=n; i > 0; i/=2) \rightarrow O(\log n)$

$\text{for}(i=1; i < n; i*=2) \rightarrow O(\log n)$

$\text{for}(i=0; i < n; i++) \rightarrow n$
 $\text{for}(j=0; j < n; j++) \rightarrow n \rightarrow O(n^2)$

$\text{for}(i=0; i < n; i++); \rightarrow n$
 $\text{for}(j=0; j < n; j++); \rightarrow n \rightarrow O(n)$

$\text{for}(i=n; i > 0; i/=2) \rightarrow \log n$
 $\text{for}(j=0; j < n; j++) \rightarrow n \rightarrow O(n \log n)$

Space complexity

- Finding approximate space requirement of the algorithm to execute inside memory

Total space

=

Input space

+

Auxiliary space

↓
actual space
required to store
input

↓
extra space
required to process
input

```
int sumOfArray (int arr[]) {  
    int sum = 0;  
    for (i = 0; i < arr.length; i++)  
        sum += arr[i];  
    return sum;  
}
```

~

input variable = arr
processing variables = sum, i

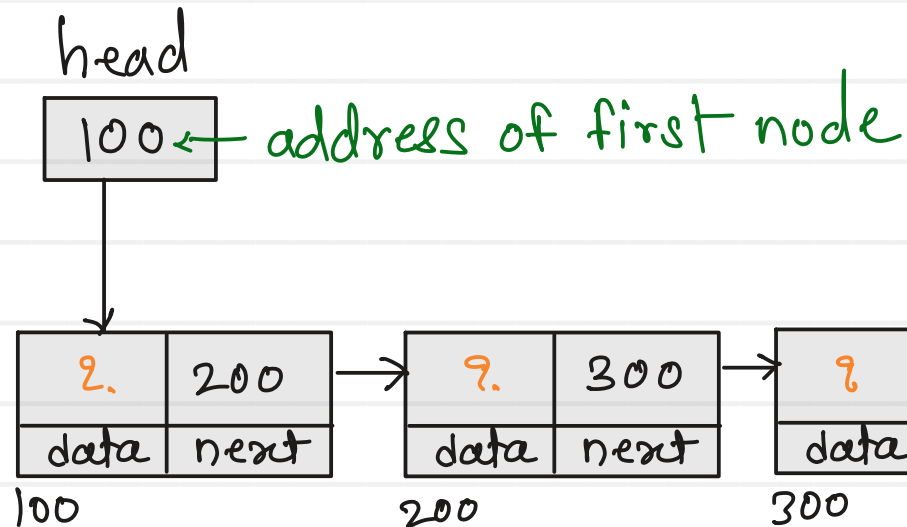
input space = n units

Auxiliary space = 2 units

Auxiliary Space will be constant for every size of array.

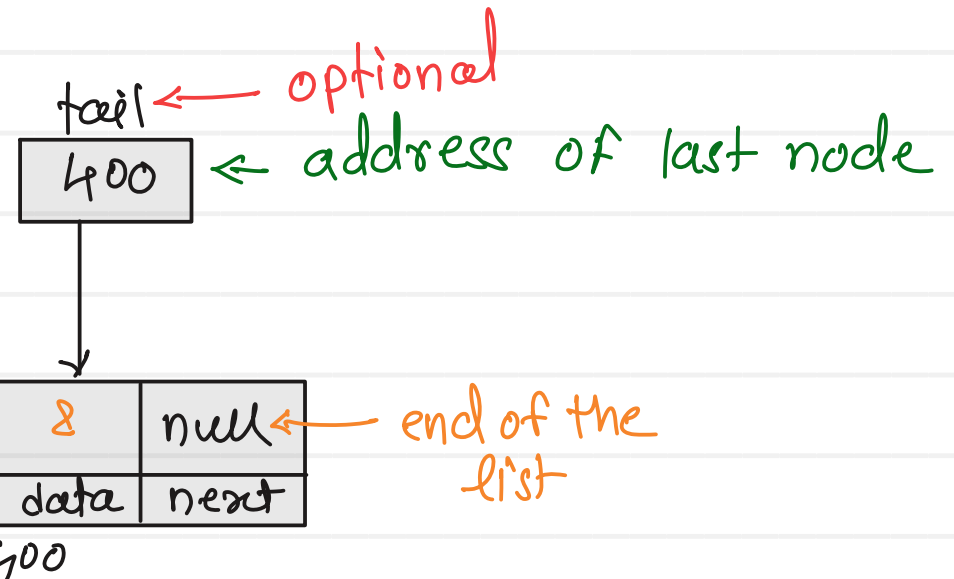
$$S(n) = O(1)$$

- linked list is a linear data structure which stores similar type of data.
- every data has address of next data with it.
- elements of linked list are referred as Node.



- Every Node will have two parts:
 data : actual value/data
 link/next : address/reference of next node

data	next



Operations

1. Add first
2. Add last
3. Add position (insert)

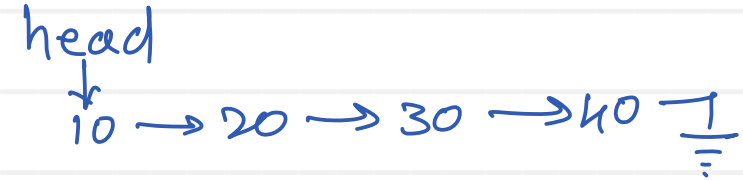
1. Delete first
2. Delete last
3. Delete position

1. Display (traverse) (forward/ backward)

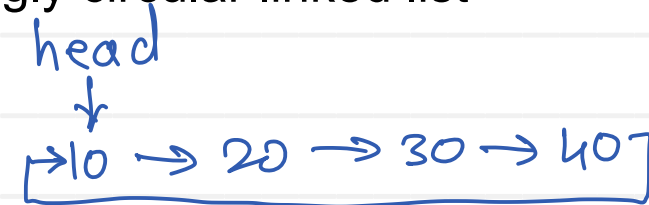
1. Search
2. Sort
3. Reverse

Types

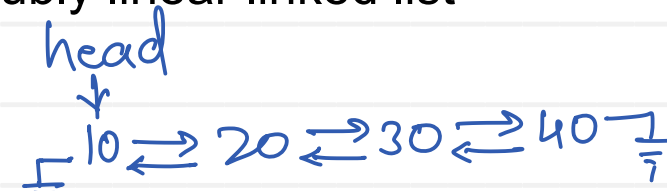
1. Singly linear linked list



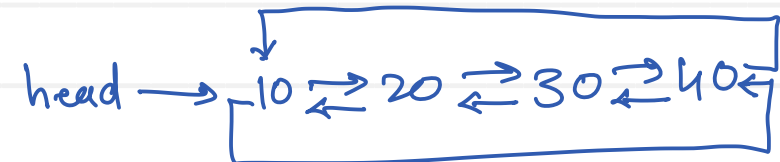
2. Singly circular linked list



3. Doubly linear linked list



4. Doubly circular linked list



Node :

data — int, char, double, enum, class

next — reference of next node

class Node { ← self referential class

int data;

Node next;

}

Why inner class ?

— to access private fields of Node class directly into LinkedList class

Why static ?

— to restrict access of ^{non static} private fields of LinkedList class into Node class

— because no dependency of Linked list class to create object of Node class

```
class LinkedList {  
    static class Node {
```

```
        int data;
```

```
        Node next;
```

```
    }
```

```
    Node head, tail;
```

```
    int count;
```

```
    public LinkedList() { ... }
```

```
    public void addNode(value) { ... }
```

```
    public void deleteNode(pos) { ... }
```

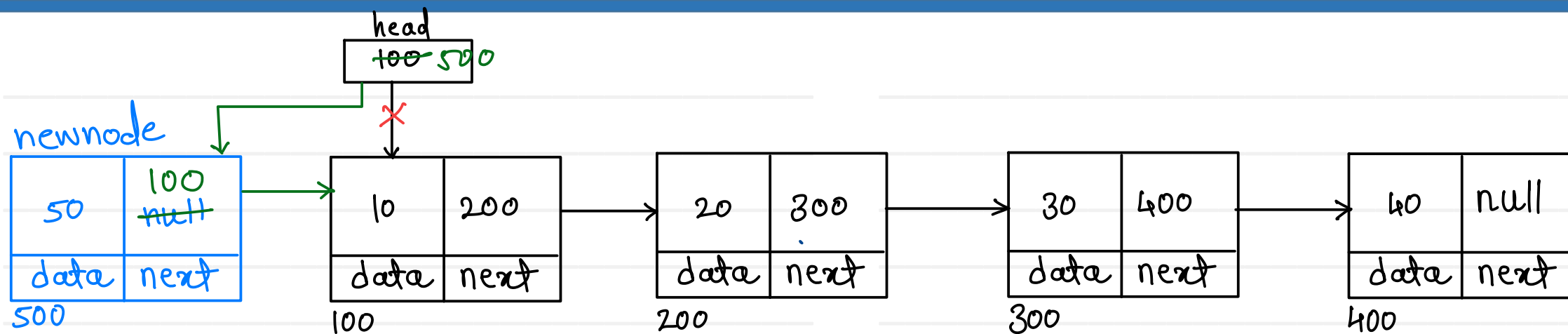
```
    public void traverse() { ... }
```

```
    public Node searchNode(key) { ... }
```

```
    public void deleteAll() { ... }
```

}

Singly linear Linked List - Add first

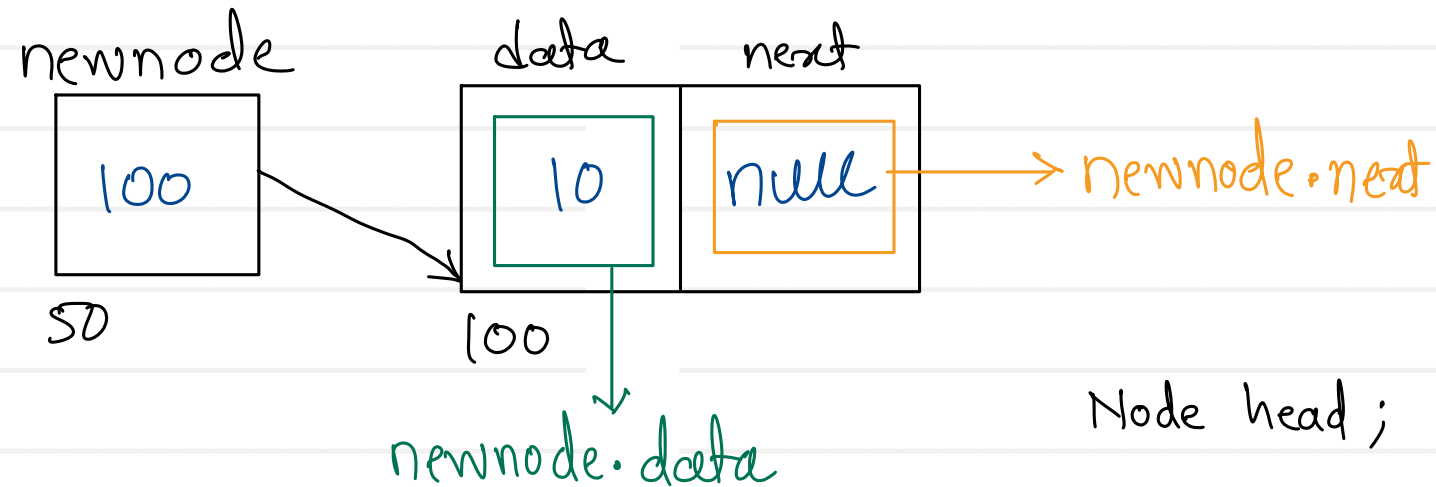


Node newnode = new Node();
 newnode.next = head;
 head = newnode;

1. Create a newnode with given data
2. add first node into next of newnode
3. move head on newnode.

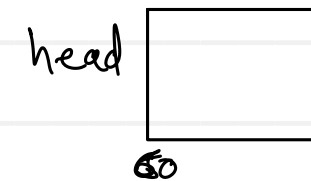
$$T(n) = O(1)$$

```
Node newnode = new Node(10);
```

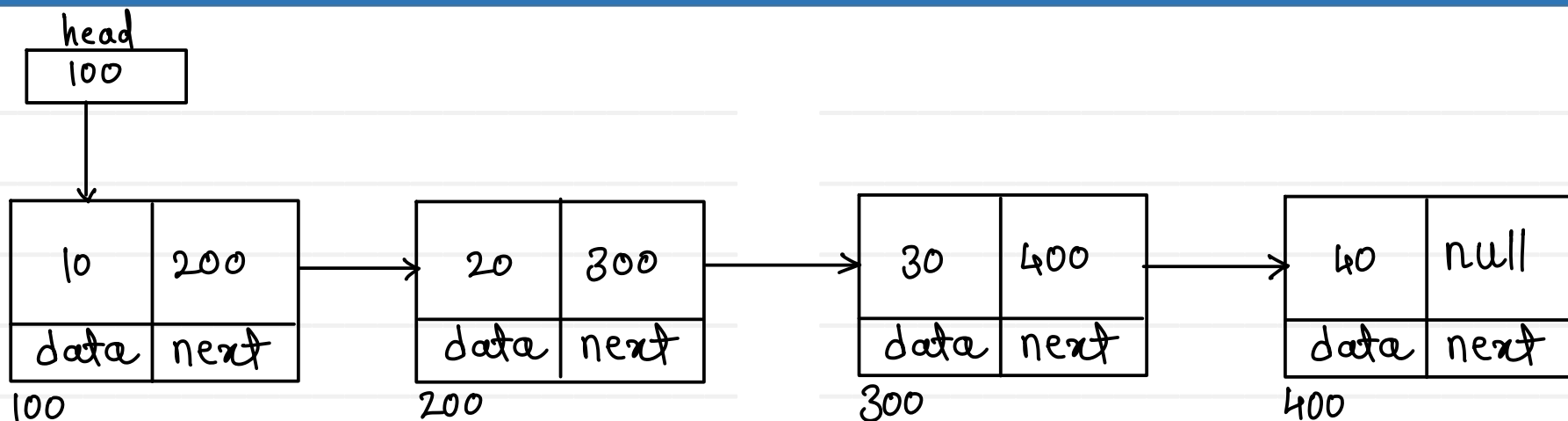


`newnode.data = 10` ← writing
`10 = newnode.data` ← reading

Node head;



Singly linear Linked List - Display



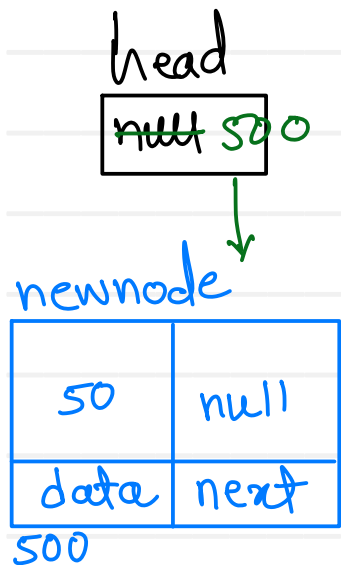
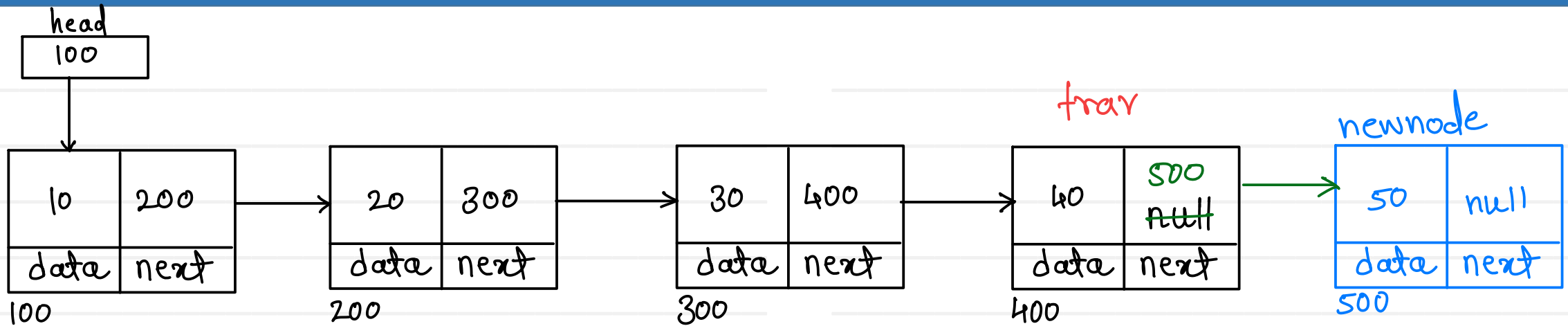
trav

1. create reference to traverse & start at first node
2. visit/print current node
3. go on next node
4. repeat step 2 & 3 till last node

$$T(n) = O(n)$$

trav	trav.data	trav.next
100	10	200
200	20	300
300	30	400
400	40	null
null		

Singly linear Linked List - Add last



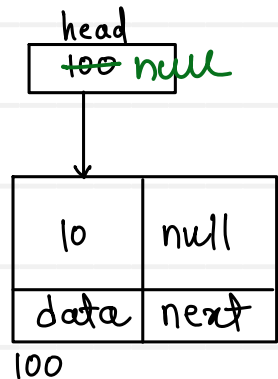
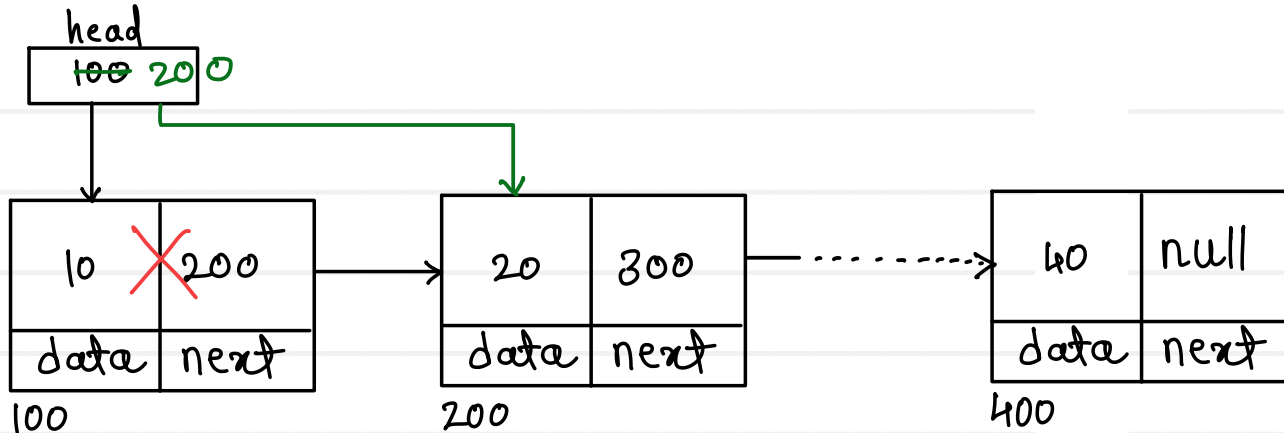
Node trav = head;
while (trav.next != null)
trav = trav.next;

1. create a newnode with given value
2. if list is empty,
add newnode into head itself.
3. if list is not empty,
a. traverse till last node
b. add newnode into next of last node

$$T(n) = O(n)$$

with tail:
 $T(n) = O(1)$

Singly linear Linked List - Delete first



$head = head.next$
 $\rightarrow$ becomes null

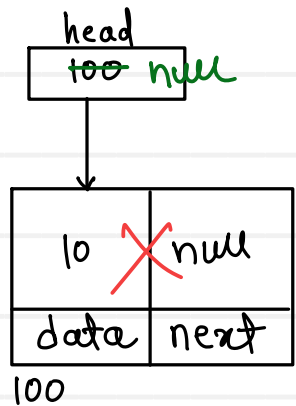
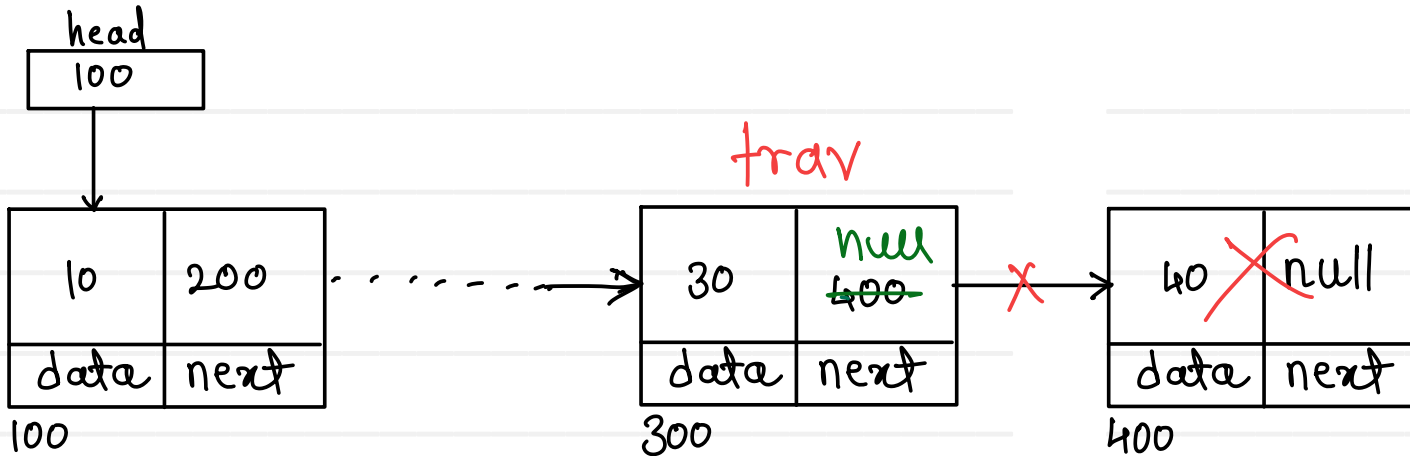


$head = head.next$
 $\downarrow$
 null pointer exception

1. if list is empty, return
2. if list is not empty,
move head on second node

$$T(n) = O(1)$$

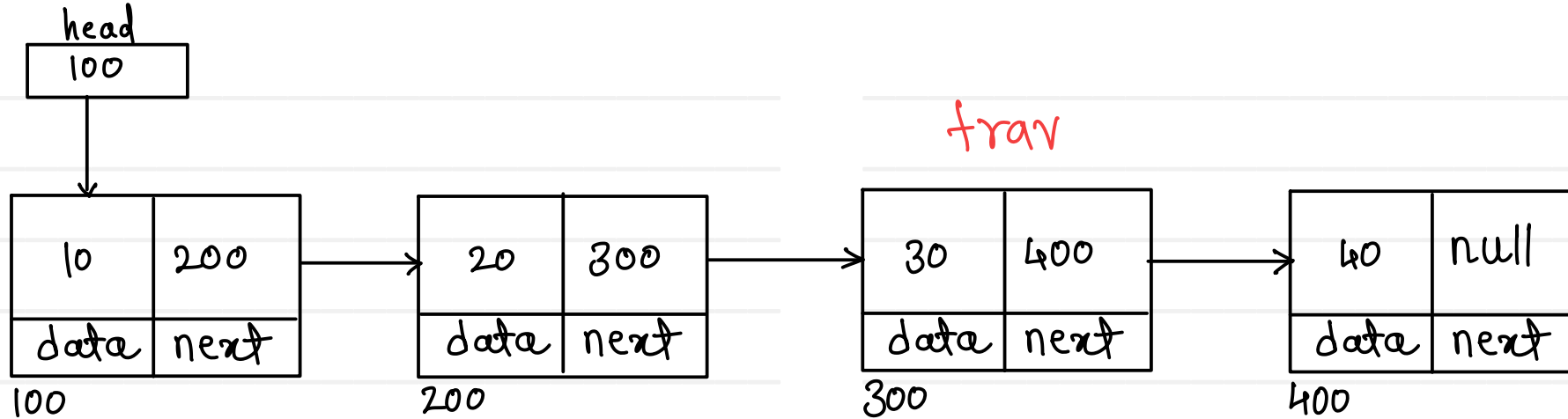
Singly linear Linked List - Delete last



```
trav = head;
while (trav.next.next != null)
    trav = trav.next;
```

1. if list is empty, return
2. if list has single node, head = null
3. if list has multiple nodes,
 - a. traverse till second last node
 - b. add null into next of second last node

$$T(n) = O(n)$$



```

trav = head;
while (trav.next.next != null)
    trav = trav.next;
    
```

trav

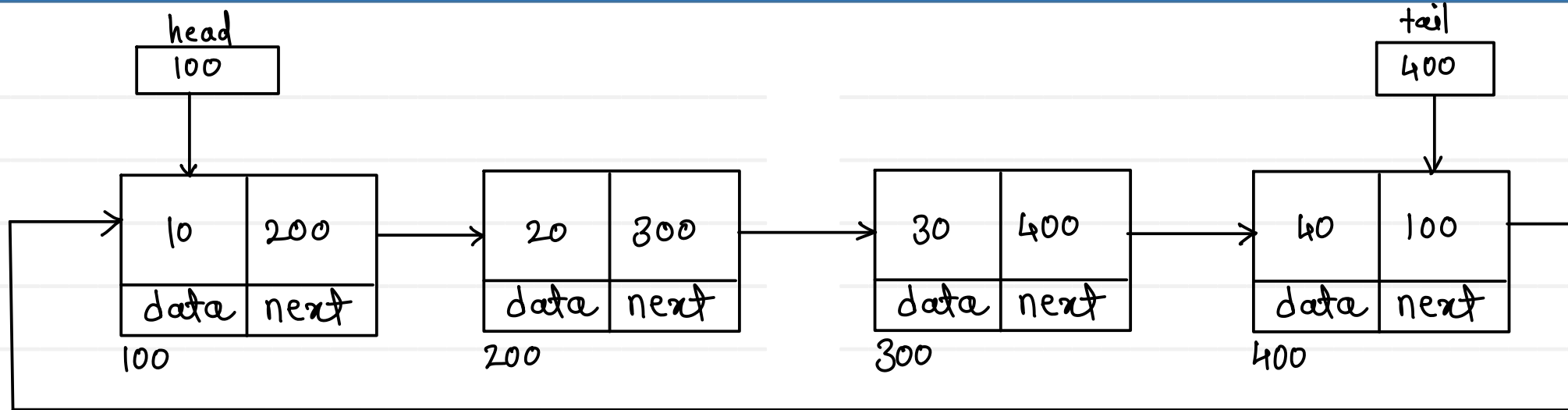
trav	trav.next	trav.next.next
100	200	300
200	300	400
300	400	null

↑ addr of second last node

```
Node trav = head;  
while (trav != null) → trav = null  
    trav = trav.next;
```

```
Node trav = head;  
while (trav.next != null) → trav = last node  
    trav = trav.next;
```

```
Node trav = head;  
while (trav.next.next != null) → trav = second last node  
    trav = trav.next;
```

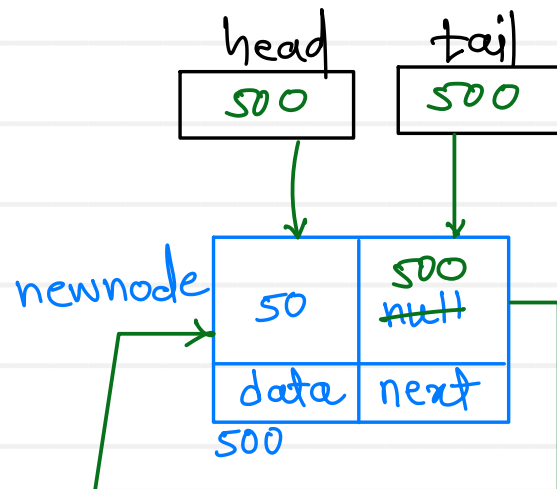
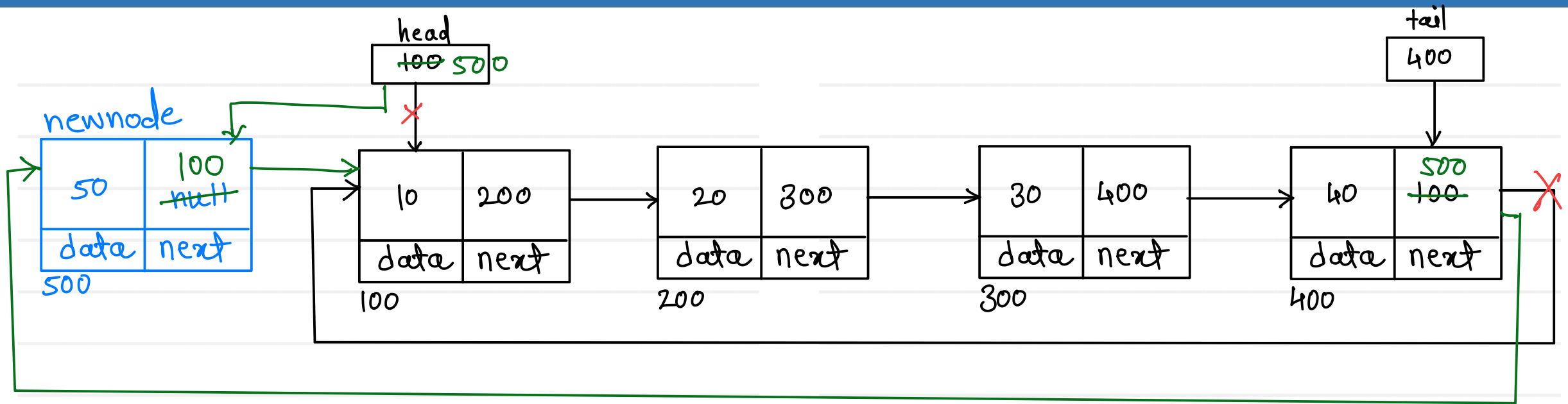


1. Create trav & start at first node
2. print current node data
3. go on next node
4. repeat above 2 steps for each node

```

Node trav = head;
do {
    cout << trav->data;
    trav = trav->next;
} while (trav != head)
    
```

Singly Circular Linked List - Add first



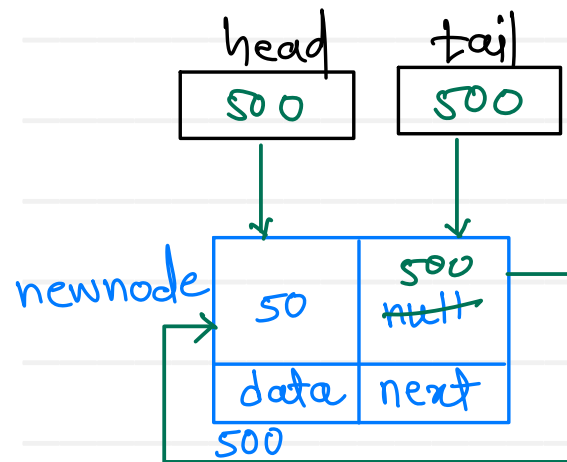
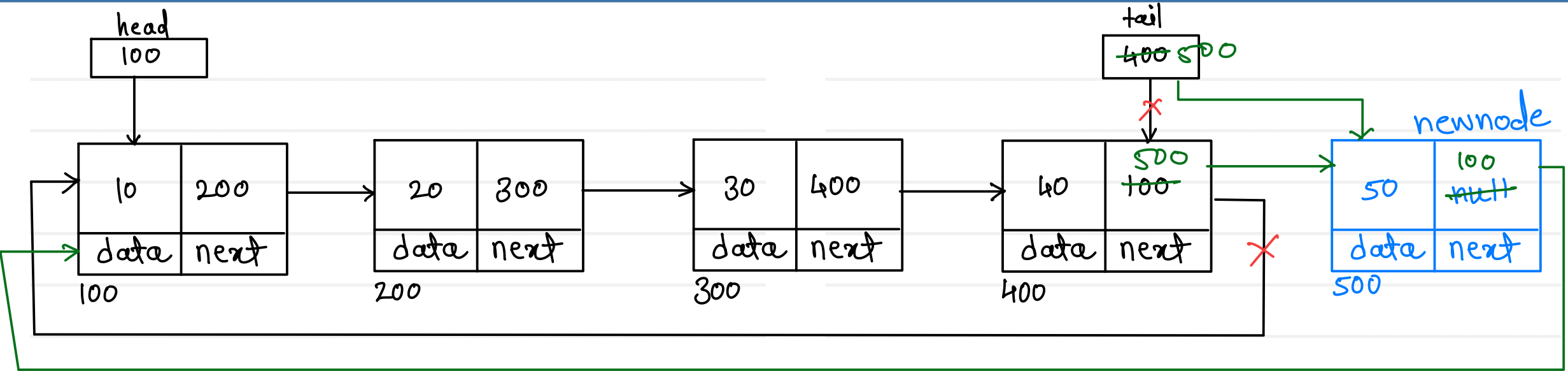
1. if list is empty,
 - a. add newnode into head & tail
 - b. make list circular

2. if list is not empty,
 - a. add first node into next of newnode
 - b. add newnode into next of last node
 - c. move head on newnode.

$$T(n) = O(1)$$

Singly Circular Linked List - Add last

make before break

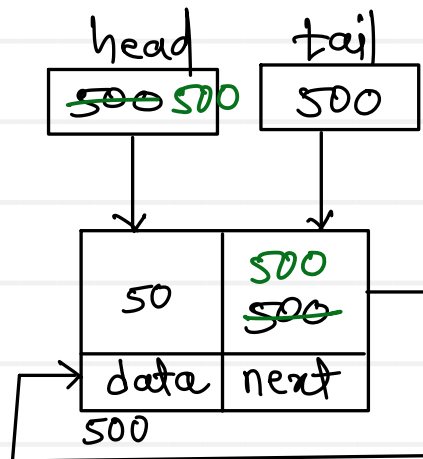
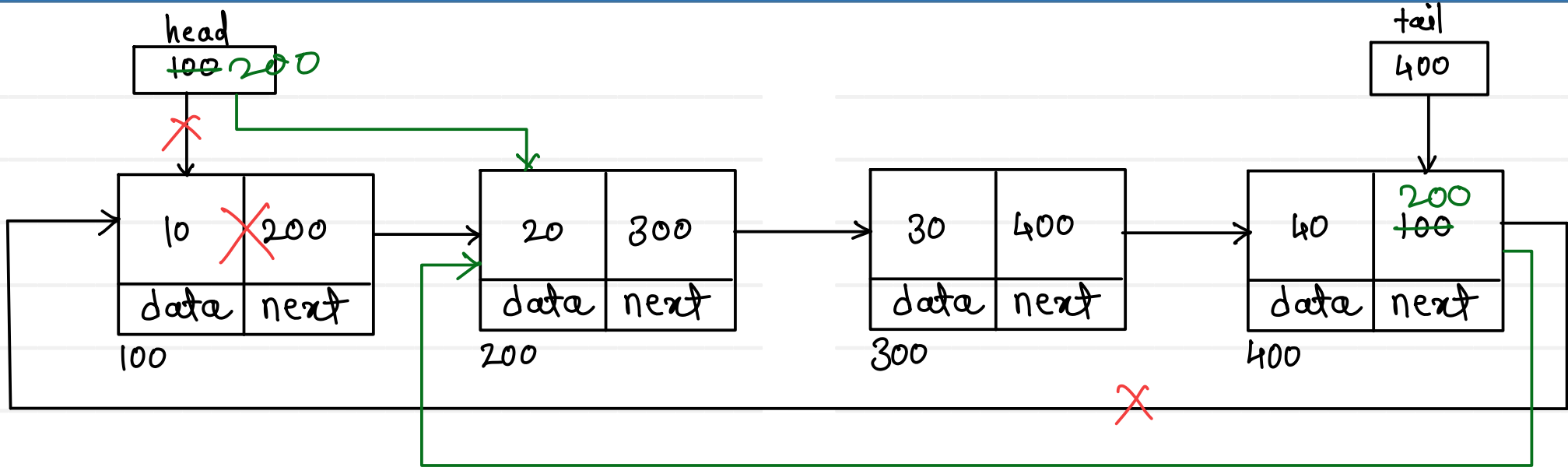


1. if list is empty,
a. add newnode into head & tail
b. make list circular

2. if list is not empty,
a. add first node into next of newnode
b. add newnode into next of last node
c. move tail on newnode.

$$T(n) = O(1)$$

Singly Circular Linked List - Delete first

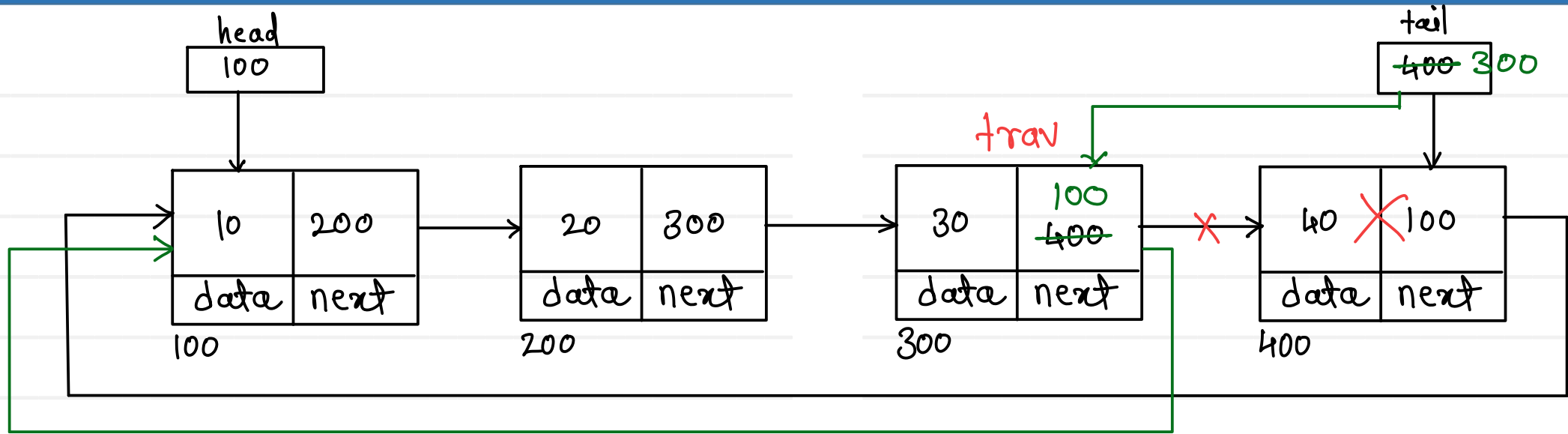


tail.next = head.next;
head = head.next;

$$T(n) = O(1)$$

1. if list is empty, return
2. if list has single node,
head = tail = null.
3. if list has multiple nodes,
 - a. add second node into next of last node
 - b. move head on second node.

Singly Circular Linked List - Delete last



```
while(trav.next.next != head)
    trav = trav.next;
```

or

```
while (trav.next != tail)
    trav = trav.next;
```

$$T(n) = O(n)$$

1. if list is empty, return
2. if list has single node
head = tail = null.
3. if list has multiple nodes
 - a. traverse till second last node
 - b. add first node into next of second last
 - c. move tail on second last node

Array

- Array space inside memory is continuous
- Array can not grow or shrink at runtime
- Random access of elements is allowed
- Insert or delete, needs shifting of array elements
- Array needs less space

Linked list

- Linked list space inside memory is not continuous
- Linked list can grow or shrink at runtime
- Random access of elements is not allowed
- Insert or delete, need not shifting of linked list elements
- Linked list needs more space